

# EduAI Platform

AI-Powered School & College Management System

*Project Documentation*

<https://github.com/Bitukumar123/EQUA-AI-PROJECT.git>

Demo vedio post in linkedln .linkedin link: <https://www.linkedin.com/in/bitukumar-a50044391>

Built with Flask · MySQL · HTML/CSS/JavaScript · OpenAI API

# 1. Introduction

EduAI Platform is a full-stack school/college management system that connects four kinds of users — the institution itself, principals, students, and parents — under one login-and-registration system, backed by a MySQL database. On top of standard functions like attendance and result tracking, the platform integrates OpenAI's GPT models to provide a conversational AI teacher for students, automatic AI-generated exams with instant grading, and a support chatbot embedded on every dashboard.

The system was built using Python (Flask) for the backend, MySQL for persistent storage, and HTML/CSS/JavaScript for the frontend — no separate frontend framework is required, keeping the stack simple to run, host, and extend.

## 1.1 Objectives

- Provide a single platform where an institution, its principals, its students, and their parents can all interact with the same underlying data.
- Give every user role a clear, separate login and registration flow, with data scoped correctly (e.g. a parent can only see their own child's records).
- Store all academic data — attendance and exam results — in a relational MySQL database for reliability and reporting.
- Bring AI into the classroom in three concrete ways: a tutoring chat assistant, automatic exam generation, and automatic grading with feedback.
- Make attendance and performance visible and easy to check for the people who need it — students checking their own record, and parents checking their child's.

## 2. User Roles & Registration Flow

The platform supports four distinct account types, each with its own registration form and login credentials:

| Role                         | Registers with                                                | Can do                                                                       |
|------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------|
| Institution (College/School) | Institution name + a self-chosen Institution ID + password    | Registers the school; sees a roster of all principals and students under it  |
| Principal                    | The Institution ID + a self-chosen Principal ID + password    | Marks daily attendance; creates AI-generated exams                           |
| Student                      | The Institution ID + a self-chosen Student ID + password      | Chats with the AI Teacher; takes AI exams; checks own attendance and results |
| Parent                       | Their child's Student ID + a self-chosen Parent ID + password | Views their child's attendance percentage and exam results (read-only)       |

*This design means an institution must register first (it is the anchor record), after which principals and students register under its Institution ID, and finally parents register by referencing their child's already-created Student ID. This mirrors how a real school onboards: the institution exists first, staff and students join it, and parents join through their children.*

## 3. System Architecture

The system follows a simple, classic three-tier architecture:

### 3.1 Client Layer (Browser)

Server-rendered HTML pages (Jinja2 templates) styled with a single stylesheet, plus small vanilla JavaScript files for the AI chat, the floating chatbot widget, and the whiteboard canvas. No JS framework or build step is required.

### 3.2 Application Layer (Flask)

The Flask app is organized into blueprints, one per role, so each area of the system owns its own routes:

- routes/auth.py — registration and login for all four roles, plus logout
- routes/admin.py — the institution's overview dashboard
- routes/principal.py — attendance marking and AI exam creation
- routes/student.py — student dashboard, AI Teacher chat, taking exams
- routes/parent.py — read-only view of a linked child's data
- utils/security.py — password hashing and a login\_required(role) decorator that protects each dashboard
- utils/ai.py — every OpenAI API call lives here: teacher replies, exam generation, exam grading, and the dashboard chatbot

### 3.3 Data Layer (MySQL)

db.py provides small helper functions (query\_one, query\_all, execute) around PyMySQL, using parameterized queries throughout to prevent SQL injection. The schema (schema.sql) defines eight tables, described in Section 4.

### 3.4 External Service (OpenAI API)

All AI functionality is isolated inside utils/ai.py. If no API key is configured, the AI features return a clear placeholder message instead of crashing, so the rest of the platform (login, attendance, dashboards) remains fully usable without an AI subscription.

## 4. Database Schema

All data is stored in a MySQL database named eduai\_platform, consisting of eight related tables:

| Table        | Purpose                                                                                           |
|--------------|---------------------------------------------------------------------------------------------------|
| institutions | One row per registered college/school; holds the Institution ID and password                      |
| principals   | Linked to an institution; holds the Principal ID and password                                     |
| students     | Linked to an institution; holds the Student ID, class name, and password                          |
| parents      | Linked to a specific student; holds the Parent ID and password                                    |
| attendance   | One row per student per date, with status (present / absent / late)                               |
| exams        | AI-generated exams — subject, topic, difficulty, and the questions stored as JSON                 |
| exam_results | A student's submitted answers, score, and AI-written feedback for one exam                        |
| chat_history | Every message exchanged between a student and the AI Teacher, used to keep conversational context |

Foreign keys tie students and principals to their institution, parents to their student, and attendance/exam results to the relevant student — so deleting an institution cascades sensibly through its data.

## 5. AI Features

### 5.1 AI Teacher

Available to students at /student/ai-teacher. The student chats freely with an AI tutor; every message and reply is saved to chat\_history so the conversation has memory across the session. A lightweight canvas 'whiteboard' mirrors the key lines of the AI's explanation, giving the sense of a teacher writing on a board while they talk.

### 5.2 AI-Generated Exams

A principal chooses a subject, topic, difficulty, and number of questions. The system prompts OpenAI to return a structured set of multiple-choice questions (with correct answers and short explanations) as JSON, which is stored in the exams table and made available to every student at that institution.

### 5.3 AI Grading & Feedback

When a student submits an exam, the platform grades it by comparing each answer to the stored correct answer (objective, deterministic scoring), then asks OpenAI to write a short, encouraging feedback summary that highlights which topics to review based on the questions the student missed.

### 5.4 Dashboard Chatbot

Every dashboard (institution, principal, student, parent) has a floating chat bubble in the corner. It answers general questions about the platform and the signed-in user's own data, using a short context string describing who is asking.

## 6. Attendance & Performance Tracking

Principals mark attendance once per day for every student in their institution through a simple checklist (checked = present, unchecked = absent). Each mark is stored as one row in the attendance table, keyed by student and date, so re-marking the same day updates rather than duplicates the record.

Both the student dashboard and the parent dashboard show:

- An attendance percentage calculated from the last 30 marked days
- A register-style ledger listing each date with a present / absent / late marker
- A table of every exam taken, its score, and the AI's written feedback

## 7. Technology Stack

| Layer           | Technology                                                                       |
|-----------------|----------------------------------------------------------------------------------|
| Backend         | Python 3, Flask 3 (blueprints, sessions)                                         |
| Database        | MySQL 8, accessed via PyMySQL                                                    |
| Frontend        | HTML5, CSS3 (custom stylesheet, no framework), vanilla JavaScript                |
| AI              | OpenAI API (gpt-4o-mini by default), used for chat, exam generation, and grading |
| Auth & security | Werkzeug password hashing, Flask sessions, role-based route protection           |

| Layer         | Technology                                                    |
|---------------|---------------------------------------------------------------|
| Configuration | python-dotenv (.env file) for secrets and connection settings |

## 8. Setup Summary

- Install dependencies: `pip install -r requirements.txt`
- Create the database and tables: `mysql -u root -p < schema.sql`
- Copy `.env.example` to `.env` and fill in `DB_PASSWORD`, `OPENAI_API_KEY`, and `SECRET_KEY`
- Run the app: `python app.py`
- Open `http://localhost:5000` and register an institution, then a principal, a student, and a parent, in that order

## 9. Future Enhancements

- Move ID collision checks and login into a single unified users table with a role column
- Add CSRF protection and rate limiting before any public deployment
- Support file/image uploads for exam questions and richer whiteboard drawing
- Add email/SMS notifications to parents when attendance or a new result is recorded
- Add pagination and search once class sizes and exam counts grow large

## 10. Conclusion

EduAI Platform demonstrates a complete, working education-management system: four role-based logins backed by a relational database, real attendance and performance tracking, and three genuinely useful AI integrations — a tutor, an exam generator, and an automatic grader — wired together through a single, easy-to-run Flask application.